

Esame OS Internals (Cabodi)

Data: 08/07/2022

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e Algoritmo PFF)

Contesto: Si consideri, per un dato processo, la seguente stringa di accessi alla memoria. Per ogni indirizzo (indirizzamento Byte, con indirizzi espressi in codice esadecimale) viene riportata anche l'operazione di lettura (R)/ scrittura (W):

R 33FB, R 2264, W 0AD3, W 237E, R 08C8, W 13D7, R 094A, R 3465, W 32A0, R 1BB0, W 0CE5, R 1480, R 3294, R 2AB8.

Supponiamo che gli indirizzi fisici e logici siano rappresentati su 16 bit, la dimensione della pagina sia 2 KByte e 5D02 sia l'indirizzo massimo utilizzabile dal programma (il limite superiore dello spazio degli indirizzi).

Si risponda alle seguenti domande:

- Calcolare la dimensione dello spazio di indirizzamento (espressa come numero di pagine) e la frammentazione interna.
- Calcolare la stringa degli accessi alle pagine.
- Simulare un algoritmo di sostituzione delle pagine di tipo PFF (Page Fault Frequency), assumendo un'unica soglia di tempo $C = 2$. Rappresentare il resident set (frame fisici contenenti pagine logiche) dopo ogni accesso alla memoria. Indicare esplicitamente le attivazioni dell'algoritmo (la procedura di selezione della vittima). Rappresentare anche (con un pedice o altra notazione) i bit di riferimento associati a pagine / frame.

L'algoritmo PFF (approssimato) può essere riassunto come segue: L'algoritmo viene attivato ad ogni page fault, non ad ogni accesso a pagina. A seconda dell'intervallo di tempo TAU dal page fault precedente:

- se $TAU < C$ (frequenza di PF maggiore di quella desiderata), aggiungere un nuovo frame al Resident Set del processo.
- se $TAU \geq C$ (frequenza di PF OK), rimuovere dal resident set tutte le pagine con bit di riferimento a 0; quindi azzerare il bit di riferimento di tutte le altre pagine nel Resident Set.

NOTA: Quando si valuta TAU , considerare solo gli istanti di tempo "tra" i due page fault (es. PF ai tempi 7 e 8 $\rightarrow TAU = 0$; PF ai tempi 12 e 15 $\rightarrow TAU = 2$). I bit di riferimento vengono posti a 1 a ciascun accesso e azzerati come ultimo passaggio dell'algoritmo. Ogni volta che i bit vengono azzerati, si include la pagina relativa al PF corrente.

- Mostrare i page fault (accessi a pagine esterne al resident set) e calcolare il loro conteggio complessivo.

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF: Mean Time To Failure), il tempo medio per riparazione (MTTR: Mean Time To Repair), e il tempo medio tra perdite di dato (MTTDL: Mean Time To Data Loss).

Si risponda alle seguenti domande:

- A causa di vincoli aggiuntivi, la probabilità di guasto di un disco, mentre l'altro è in riparazione, viene RADDOPPIATA. Si può considerare che questo sia equivalente a:
 - A1) moltiplicare MTTR per 2?
 - A2) moltiplicare MTTR per 0.5?
 - A3) Lasciare tutto com'è (senza modifiche) in quanto MTTR è trascurabile?
 - Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTR = 36$ ore, quanto deve essere MTTF per garantire $MTTDL > 100 \cdot MTTF$?
-

Domanda 3 (Ottimizzazioni Memoria Virtuale)

Contesto: Si considerino le ottimizzazioni implementate per migliorare le prestazioni complessive della gestione della memoria virtuale.

Si risponda alle seguenti domande:

- Spiegare brevemente la tecnica COW (Copy on Write): che cosa è e come migliora le prestazioni della gestione della memoria? I miglioramenti sono in termini di tempo? Di occupazione di memoria? Di entrambi tempo e memoria? (Motivare).
- Spiegare perché l'I/O sulla partizione di swap (backing store) è più veloce dell'I/O mediante file system (anche se a parità di disco). E' possibile che lo spazio di indirizzamento virtuale di un processo sia più grande del numero di blocchi totali disponibili su backing store? (Motivare).
- Quale è il ruolo del bit di validità (valid/invalid bit) associato a una pagina? Il validity bit viene memorizzato solo nella tabella delle pagine oppure viene anche immagazzinato nella TLB (quando disponibile su un dato processore)? (Motivare).

SECTION 2: OS161

Domanda 4 (Gestione Console e Sincronizzazione)

Contesto: Si consideri la gestione della console in OS161 e le funzioni scritte di seguito.

```
void putch_intr (struct con_softc *cs, int ch) {
    P(cs->cs_wsem);
    cs->cs_send(cs->cs_devdata, ch);
}
```

```
void con_start(void *vcs) {
    struct con_softc *cs = vcs;
    V(cs->cs_wsem);
}
```

Si risponda alle seguenti domande:

- Siccome `putch_intr` è chiamata da `putch(int ch)`, e si tratta di un output (del carattere `ch`), perché `putch_intr` aspetta (chiamando P) invece di segnalare (V)?
 - Fornire la definizione del campo `cs_wsem` nella `struct con_softc`.
 - Fornire un'implementazione delle due funzioni, sostituendo il semaforo `cs_wsem` con una condition variable (definire anche i campi necessari in `struct con_softc`).
-

Domanda 5 (Strutture Processi e Thread)

Contesto: Sia dato un sistema OS161, nella versione base. Si considerino le definizioni delle `struct thread` e `struct proc` parzialmente qui riportate:

```
struct thread {
    char *t_name;
    void *t_stack;
    struct switchframe *t_context;
    struct cpu *t_cpu;
    struct proc *t_proc;
    /* add more here as needed */
};
```

```
struct proc {
    unsigned p_numthreads;
    struct addrspace *p_addrspace;
    struct vnode *p_cwd;
    /* add more here as needed */
};
```

Si risponda alle domande che seguono:

- Si dica come sono correlate/collegate le due struct nel caso in cui un processo abbia più di un thread.

- Si dica anche se, dato un puntatore `p` a una `struct proc`, sia possibile ricavare l'elenco di tutti i thread del processo e stamparne il nome. Qualora non sia possibile, si proponga come modificare (aggiungere campi) a `struct proc` e/o `struct thread`, in modo da poter realizzare l'operazione (dato un processo, elencare i suoi thread). Scrivere la funzione `printThreadNames(struct proc *p)` che svolge tale compito.
- Si spieghi brevemente il ruolo del puntatore `t_context` nella `struct thread`.

ANSWERS

Domanda 1 (Memoria Virtuale e Algoritmo PFF)

Dati Iniziali:

- Dimensione pagina: $2 \text{ KB} = 2048 \text{ Byte} = 2^{11} \text{ Byte}$ (Offset di 11 bit).
- Indirizzi su 16 bit.
- Indirizzo massimo: `0x5D02`.

A) Spazio di Indirizzamento e Frammentazione:

L'indirizzo massimo logico è `0x5D02`, che in decimale corrisponde a 23810. I byte allocati dal processo sono quindi 23811 (da 0 a 23810).

$$\text{Pagine del processo} = \lceil \frac{23811}{2048} \rceil = \mathbf{12 \text{ pagine.}}$$

Lo spazio di indirizzamento effettivo del processo occupa 12 pagine (dalla pagina logica 0 alla pagina logica 11).

La frammentazione interna si calcola unicamente sull'ultima pagina allocata:

$$\text{Spazio Fisico Allocato} = 12 \times 2048 = 24576 \text{ Byte.}$$

$$\text{Frammentazione Interna} = 24576 - 23811 = \mathbf{765 \text{ Byte.}}$$

B) Stringa degli accessi alle pagine:

Per ottenere il Virtual Page Number (VPN), si scorre l'indirizzo a destra di 11 bit (oppure si divide l'indirizzo decimale per 2048 troncando la virgola).

- `33FB` (13307) → **Pag 6**
- `2264` (8804) → **Pag 4**
- `0AD3` (2771) → **Pag 1**
- `237E` (9086) → **Pag 4**
- `08C8` (2248) → **Pag 1**
- `13D7` (5079) → **Pag 2**
- `094A` (2378) → **Pag 1**
- `3465` (13413) → **Pag 6**
- `32A0` (12960) → **Pag 6**
- `1BB0` (7088) → **Pag 3**

- 0CE5 (3301) → **Pag 1**
- 1480 (5248) → **Pag 2**
- 3294 (12948) → **Pag 6**
- 2AB8 (10936) → **Pag 5**

Stringa accessi: **6, 4, 1, 4, 1, 2, 1, 6, 6, 3, 1, 2, 6, 5.**

C) Simulazione Algoritmo PFF ($C = 2$):

Si riporta il Resident Set al termine di ogni istante t (la notazione $P(b)$ indica la pagina P con bit di riferimento b).

Regole di base:

- Se $\tau < 2$: Si aggiunge la nuova pagina al Resident Set.
- Se $\tau \geq 2$: Si eliminano le pagine con $b = 0$, si inserisce la nuova, e **si azzerano i bit di tutte le restanti**.

t	Pag.	Esito	τ	Azione Algoritmo PFF	Resident Set (Pag(Bit))
1	6	PF	-	Aggiungo 6	6(0)
2	4	PF	0	$\tau < 2$: Aggiungo 4	6(0), 4(0)
3	1	PF	0	$\tau < 2$: Aggiungo 1	6(0), 4(0), 1(0)
4	4	Hit	-	Imposto bit di 4 a 1	6(0), 4(1), 1(0)
5	1	Hit	-	Imposto bit di 1 a 1	6(0), 4(1), 1(1)
6	2	PF	2	$\tau \geq 2$: Rimuovo 6(0). Aggiungo 2(0). Azzero i bit.	4(0), 1(0), 2(0)
7	1	Hit	-	Imposto bit di 1 a 1	4(0), 1(1), 2(0)
8	6	PF	1	$\tau < 2$: Aggiungo 6	4(0), 1(1), 2(0), 6(0)
9	6	Hit	-	Imposto bit di 6 a 1	4(0), 1(1), 2(0), 6(1)
10	3	PF	1	$\tau < 2$: Aggiungo 3	4(0), 1(1), 2(0), 6(1), 3(0)
11	1	Hit	-	Imposto bit di 1 a 1	4(0), 1(1), 2(0), 6(1), 3(0)
12	2	Hit	-	Imposto bit di 2 a 1	4(0), 1(1), 2(1), 6(1), 3(0)
13	6	Hit	-	Imposto bit di 6 a 1	4(0), 1(1), 2(1), 6(1), 3(0)
14	5	PF	3	$\tau \geq 2$: Rimuovo 4(0), 3(0). Aggiungo 5(0). Azzero i bit.	1(0), 2(0), 6(0), 5(0)

Totale Page Faults: 7.

Domanda 2 (RAID e Affidabilità)

A) Effetto del raddoppio della probabilità di guasto:

La risposta corretta è la **A1) Moltiplicare MTTR per 2 (VERO)**.

Motivazione: La probabilità P che il disco superstita si guasti durante la finestra di riparazione del disco guasto è data dal rapporto tra la durata della riparazione e il tempo medio di vita del disco:

$$P = \frac{MTTR}{MTTF}$$

Se il testo impone che tale probabilità venga raddoppiata a causa di vincoli aggiuntivi, otteniamo:

$$P_{nuova} = 2 \times \left(\frac{MTTR}{MTTF} \right) = \frac{2 \times MTTR}{MTTF}$$

Dal punto di vista matematico e modellistico, questo equivale esattamente a considerare un sistema in cui il tempo medio di riparazione ($MTTR$) è grande il doppio. Le opzioni A2 (dimezzare) e A3 (ignorare) sono conseguentemente **FALSE**.

B) Calcolo del MTTF per garantire il vincolo di affidabilità:

In una configurazione RAID 1 (Mirrored) composta da 2 dischi indipendenti, la formula teorica per il Mean Time To Data Loss (MTTDL) incorpora già il fattore 2 al denominatore (derivante dal fatto che il tempo atteso per il primo guasto nel sistema a due dischi è $MTTF/2$):

$$MTTDL = \frac{MTTF^2}{2 \times MTTR}$$

Il testo richiede di soddisfare il seguente vincolo di affidabilità:

$$MTTDL > 100 \times MTTF$$

Sostituendo la formula del MTTDL nella disequazione:

$$\frac{MTTF^2}{2 \times MTTR} > 100 \times MTTF$$

Dividendo entrambi i membri per $MTTF$ (assumendo $MTTF > 0$):

$$\frac{MTTF}{2 \times MTTR} > 100$$

Isolando l'incognita $MTTF$:

$$MTTF > 100 \times (2 \times MTTR)$$

$$MTTF > 200 \times MTTR$$

Essendo noto dai dati che $MTTR = 36$ ore:

$$MTTF > 200 \times 36$$

$$MTTF > \mathbf{7200} \text{ ore}$$

Affinché il sistema rispetti il vincolo richiesto, il tempo medio tra i guasti di un singolo disco deve essere strettamente superiore a 7200 ore.

Domanda 3 (Ottimizzazioni Memoria Virtuale)

A) Tecnica Copy-On-Write (COW):

La tecnica COW ottimizza la creazione di nuovi processi (come la system call `fork()` in UNIX) permettendo al processo padre e al processo figlio di condividere inizialmente le stesse pagine in memoria fisica. Le pagine condivise vengono marcate in sola lettura. Solo quando uno dei due processi tenta di eseguire una scrittura (Write) su una di queste pagine, il sistema operativo intercetta l'eccezione, alloca un nuovo frame fisico, vi copia il contenuto della pagina originale e aggiorna la tabella delle pagine del processo scrivente (Copy).

I miglioramenti avvengono in **entrambi** i domini:

- **Tempo:** La `fork()` risulta quasi istantanea, poiché si evita la latenza derivante dalla copia fisica e preventiva dell'intero spazio di indirizzamento (spesso di svariati MB o GB).

- **Occupazione di memoria:** Molte pagine (come il segmento codice o dati in sola lettura) non verranno mai modificate. La COW permette di mantenere queste pagine condivise indefinitamente, con un enorme risparmio di memoria RAM fisica.

B) I/O su Backing Store vs File System e Dimensione VAS:

Velocità dello Swap: L'I/O sul backing store (partizione di swap) è significativamente più veloce perché salta completamente lo stack software del File System. Il File System introduce sovraccarico (overhead) per gestire metadati, i-node, directory e frammentazione dei file. Lo swap, invece, è gestito tipicamente come un blocco contiguo (raw block device), permettendo accessi diretti e massivi.

VAS > Backing Store: **SÌ, è assolutamente possibile.** Lo Spazio di Indirizzamento Virtuale (VAS) di un processo è composto sia da *memoria anonima* (heap, stack, dati modificati) sia da *memoria file-backed* (codice eseguibile, librerie condivise, memory-mapped files). Solo la memoria anonima necessita di essere salvata nel backing store in caso di *swap-out*. Le pagine file-backed, se necessarie altrove, vengono semplicemente scartate dalla RAM dal sistema operativo, in quanto la loro copia originale risiede già sul file system. Pertanto, un processo può mappare in memoria file per svariati Gigabyte, superando la dimensione totale dello swap, pur utilizzandone una minima frazione.

C) Ruolo del Valid/Invalid Bit in PT e TLB:

Il bit di validità svolge funzioni cruciali ma leggermente differenti nei due contesti:

- **Nella Page Table (PT):** Indica se una pagina virtuale appartiene legalmente allo spazio di indirizzamento del processo e se il frame corrispondente è attualmente caricato in memoria RAM. Un bit invalido scatena un'eccezione hardware (Page Fault o Segmentation Fault).
- **Nella TLB:** Il bit di validità è **sempre presente** e serve a indicare se l'entry della cache TLB contiene una traduzione affidabile. *Motivazione:* Essendo la TLB una memoria associativa hardware (ricerca parallela in $O(1)$, non lineare), il bit è essenziale per la gestione dei **Context Switch**. Quando la CPU passa a eseguire un nuovo processo, le traduzioni virtuale-fisico del processo precedente diventano errate per il nuovo contesto. Il sistema operativo esegue un *TLB Flush* semplicemente forzando a 0 i bit di validità di tutte le entry, rendendole invalide senza doverne cancellare il contenuto.

Domanda 5 (Strutture Processi e Thread)

A) Correlazione tra le struct nel caso di processi multi-thread:

In OS161, il processo (`struct proc`) agisce come un puro "contenitore" di risorse, mentre i thread (`struct thread`) rappresentano i flussi di esecuzione. Quando un processo ha più thread, la correlazione avviene in questo modo:

- **Condivisione Risorse:** Tutti i thread appartenenti allo stesso processo **condividono** la stessa memoria e le stesse risorse. Non vi è alcuna Copy-On-Write (COW), poiché tutti i thread accedono simultaneamente e direttamente allo stesso `p_addrspace` (spazio di indirizzamento virtuale) e allo stesso `p_cwd` (current working directory) del processo genitore.
- **Indipendenza di Esecuzione:** Ogni thread mantiene il proprio stack di esecuzione indipendente (`t_stack`).
- **Puntatori di collegamento:** Ogni thread mantiene un puntatore al processo a cui appartiene tramite il campo `t_proc`. Il processo, a sua volta, tiene traccia del numero totale di thread attivi al suo interno aggiornando la variabile `p_numthreads`.

B) Elenco dei thread e funzione printThreadNames:

Con le sole definizioni fornite nella traccia, **NON è possibile** ricavare l'elenco dei thread a partire dal processo. Il collegamento è unidirezionale: il thread conosce il processo (`t_proc`), ma il processo non ha puntatori ai suoi thread (ne conosce solo il numero tramite `p_numthreads`).

Modifica proposta: Per rendere possibile l'operazione, occorre aggiungere a `struct proc` una struttura dati per tracciare i thread. Possiamo aggiungere un array statico di puntatori (o in alternativa, per maggiore flessibilità, una linked list di thread):

```
struct proc {
    unsigned p_numthreads;
    struct addrspace *p_addrspace;
    struct vnode *p_cwd;
    /* --- CAMPO AGGIUNTO --- */
    struct thread *p_threads[MAX_THREADS];
};
```

Tale array dovrà essere inizializzato a NULL alla creazione del processo e aggiornato (inserendo/rimuovendo il puntatore `t`) ogni volta che un thread viene creato o distrutto.

Implementazione della funzione:

```
void printThreadNames(struct proc *p) {
    /* Controllo di validità sul parametro in ingresso */
    if (p == NULL) {
        return;
    }

    kprintf("Thread appartenenti al processo:\n");

    /* Scansione dell'array per stampare i nomi dei thread */
    for (int i = 0; i < MAX_THREADS; i++) {
        if (p->p_threads[i] != NULL) {
            /* Verifica che il nome non sia nullo prima di stamparlo */
            if (p->p_threads[i]->t_name != NULL) {
                kprintf("- %s\n", p->p_threads[i]->t_name);
            } else {
                kprintf("- (unnamed thread)\n");
            }
        }
    }
}
```

C) Ruolo del puntatore t_context:

Il campo `struct switchframe *t_context` è il cuore del meccanismo di **Context Switch** (cambio di contesto).

Esso punta all'indirizzo di memoria (generalmente situato sullo stack del kernel del thread stesso) in cui è stata salvata la "fotografia" dello stato della CPU l'ultima volta che il thread è stato prelazionato (*deschedulato*). Questa struttura (`switchframe`) contiene i valori dei registri macchina necessari per riprendere l'esecuzione (Instruction Pointer, Stack Pointer, registri *callee-save*).

Quando lo scheduler decide di rimettere in esecuzione questo thread, il codice assembly di OS161 utilizzerà il puntatore `t_context` per ripristinare tutti i registri hardware della CPU, permettendo al thread di riprendere l'esecuzione esattamente dal punto in cui era stato interrotto.